Adaptive Additive Quantization for Efficient Vector DBMS

Astitwa Saxsena (2025MCS3005)
Abhishek Gupta(2025MCS2963)
Rehman (2025MCS)
Department of Computer Science and Engineering
IITD

April 18, 2026

Abstract

Modern database systems increasingly rely on vector similarity search for applications such as semantic search, recommendation systems, and AI retrieval. However, exact nearest neighbor search is computationally expensive. This project proposes a hybrid Vector DBMS architecture combining IVF indexing, Additive Quantization (AQ), and Adaptive Reranking. The system achieves high accuracy (up to 99% Recall@10) while maintaining scalable query performance.

1 Introduction

Traditional DBMS systems are designed for structured data using B-Trees and hash indexes. However, modern applications use **high-dimensional embeddings** (e.g., 128D SIFT, 784D MNIST, or transformer embeddings), where classical indexing fails due to the **curse of dimensionality**.

To solve this, modern systems like FAISS use Approximate Nearest Neighbor (ANN) search.

Problem:

- Exact search is too slow for large datasets
- ANN is fast but reduces accuracy

Goal of this project: Design a DBMS-style retrieval system that balances:

- Speed (low latency)
- Accuracy (high recall)
- Scalability (million-scale data)

Intuition (Simple View): We can think of vector search like searching for similar images or words. Instead of checking every item in the database (which is slow), we:

- First narrow down the search space (IVF)
- Then compress data for fast comparison (AQ)
- Finally refine only uncertain results (Adaptive Reranking)

This is similar to how a DBMS query optimizer avoids scanning the full table.

2 System Architecture

Our system is a **3-layer DBMS-inspired pipeline**:

2.1 Layer 1: IVF Indexing (Query Pruning)

We divide the dataset into clusters (called inverted lists). Each vector belongs to one cluster.

At query time, we do NOT search the entire dataset. Instead:

- Find the closest clusters to the query
- Search only inside those clusters

Parameter: *nprobe*
nprobe controls how many clusters we search:

- Small *nprobe* $\Rightarrow$ faster but less accurate
- Large *nprobe* $\Rightarrow$ slower but more accurate

Analogy: Like searching books in a library:

- Instead of scanning all shelves, you pick a few relevant sections

Benefit: Reduces search complexity from $O(N)$ to approximately:

$$O(\frac{N}{N_{list}})$$

2.2 Layer 2: Additive Quantization (Storage Compression)

Instead of storing full vectors, we approximate each vector using a sum of smaller codewords:

$$x \approx \sum_{i=1}^{M} C_i$$

Intuition: Think of representing a color using a mix of basic colors. Instead of storing the full color value, we store indices of base colors.

Why AQ is better than PQ:

- PQ splits vector into independent parts (loses relationships)
- AQ models dependencies between parts
- This results in lower reconstruction error and better accuracy

2.3 Layer 3: Adaptive Reranking (Query Optimization)

This is the key idea of our system.

After retrieving approximate nearest neighbors, we measure how confident the system is.

We define a confidence gap:

$$\Delta = d_2 - d_1$$

where d_1 and d_2 are distances of the top results.

Intuition:

- If the best result is much better than the second best (large gap), we are confident.
- If both are very close (small gap), the result is ambiguous.

Adaptive Strategy:

- Confident query $\Rightarrow$ return results immediately (fast)
- Ambiguous query $\Rightarrow$ fetch more candidates and rerank using exact distances

Why this works:

- Easy queries are answered quickly
- Hard queries get extra computation

Analogy: Like a student:

- If the answer is obvious, respond immediately
- If unsure, spend more time verifying

3 Limitations of Baselines

PQ:

- High compression error
- Poor accuracy in high dimensions

OPQ:

- Slight improvement over PQ
- Still independent subspace assumption

Key Insight: Neither PQ nor OPQ adapts dynamically to query difficulty.

4 Complexity Analysis

Baseline PQ/OPQ:

$$O(nprobe \cdot M)$$

AQ: Higher computation due to residual encoding:

$$O(nprobe \cdot M^2)$$

Adaptive Reranking: Worst case:

$$O(k \cdot \log n)$$

But average case is much lower because reranking is triggered only for ambiguous queries.

Interpretation:

- PQ/OPQ are fast because they use simple distance approximations
- AQ is slower due to more complex encoding
- Adaptive reranking increases cost only for difficult queries

Thus, the system achieves a balance:

- Fast average-case performance
- High worst-case accuracy

5 Experimental Results and Observations

We evaluate our system on two datasets:

- SIFT-1M (128-dimensional, large-scale benchmark)
- MNIST (784-dimensional, high-dimensional dataset)

Table 1: SIFT-1M Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.3149	0.2120	0.38 ms
OPQ	8	0.3077	0.2050	0.38 ms
AQ	8	0.4035	0.2966	15.84 ms
AQ+Adaptive	8	**0.8274**	**0.8274**	26.38 ms
PQ	16	0.5489	0.4609	0.63 ms
OPQ	16	0.5128	0.4155	0.63 ms
AQ	16	0.6002	0.5194	19.48 ms
AQ+Adaptive	16	**0.9702**	**0.9702**	43.19 ms

5.1 SIFT-1M Results

Observations:

- PQ and OPQ perform poorly at large scale, with recall below 55%.
- AQ improves recall due to better vector reconstruction.
- AQ+Adaptive dramatically boosts recall (up to 97%), nearly matching exact search.
- Increasing M improves accuracy but increases latency.
- The latency increase in AQ+Adaptive is due to reranking overhead.

Key Insight: Adaptive reranking successfully recovers information lost during quantization, making AQ competitive with exact search.

5.2 MNIST Results

Table 2: MNIST Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.4893	0.3744	0.013 ms
OPQ	8	0.5725	0.4754	0.014 ms
AQ	8	0.5961	0.5031	0.86 ms
AQ+Adaptive	8	**0.9536**	**0.9536**	1.90 ms
PQ	16	0.6194	0.5341	0.022 ms
OPQ	16	0.6926	0.6271	0.023 ms
AQ	16	0.7337	0.6796	1.64 ms
AQ+Adaptive	16	**0.9936**	**0.9936**	11.41 ms

Observations:

- OPQ performs better than PQ due to rotation optimization.
- AQ improves further due to better encoding of high-dimensional data.
- AQ+Adaptive achieves near-perfect recall (99.36%).
- Latency remains manageable compared to SIFT due to smaller dataset size.

Key Insight: High-dimensional structured data (like MNIST) benefits more from AQ and adaptive refinement.

5.3 GloVe (Word Embeddings)

We evaluate our system on GloVe embeddings (128-dimensional), which represent semantic relationships between words.

Table 3: GloVe Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.1301	0.0700	0.071 ms
OPQ	8	0.1982	0.1201	0.070 ms
AQ	8	0.2986	0.2033	0.193 ms
AQ+Adaptive	8	**0.5648**	**0.5648**	0.240 ms
PQ	16	0.3009	0.2044	0.119 ms
OPQ	16	0.3934	0.2898	0.117 ms
AQ	16	0.4810	0.3849	0.212 ms
AQ+Adaptive	16	**0.7910**	**0.7910**	0.357 ms

Observations:

- Overall recall is lower compared to SIFT and MNIST.
- This is because GloVe embeddings are more noisy and less structured.
- PQ performs poorly due to high quantization error.
- OPQ improves performance by learning a rotation.
- AQ significantly improves recall by modeling dependencies.
- AQ+Adaptive nearly doubles recall compared to AQ.

Key Insight: Semantic embeddings (like GloVe) are harder for ANN search because distances between vectors are less well-separated. Adaptive reranking helps resolve ambiguous cases.

5.4 Deep1B (Large-Scale Dataset)

We evaluate on Deep1B-style features (96-dimensional) with nearly 10 million vectors.

Observations:

Table 4: Deep Dataset Performance Comparison

Method	M	Recall	MAP	Latency
PQ	8	0.1084	0.0566	1.729 ms
OPQ	8	0.1870	0.1064	1.749 ms
AQ	8	0.3260	0.2219	2.866 ms
AQ+Adaptive	8	**0.6427**	**0.6427**	4.407 ms
PQ	16	0.3264	0.2233	3.042 ms
OPQ	16	0.3960	0.2895	3.042 ms
AQ	16	0.5346	0.4403	3.111 ms
AQ+Adaptive	16	**0.8823**	**0.8823**	7.023 ms

- This dataset is significantly larger (10M vectors), making the problem harder.
- PQ struggles due to high compression error.
- OPQ provides moderate improvement.
- AQ performs significantly better due to better approximation.
- AQ+Adaptive achieves high recall (up to 88%) even at large scale.
- Latency increases due to larger search space and reranking.

Key Insight: As dataset size increases, approximate errors accumulate. Adaptive reranking becomes more important to recover lost accuracy.

5.5 Overall Observations

- PQ is fastest but least accurate
- OPQ provides moderate improvement
- AQ significantly improves recall
- AQ+Adaptive achieves near-exact accuracy
- Trade-off exists between latency and accuracy

Observation: MAP ≈ Recall

We observe that MAP and Recall are nearly identical in AQ+Adaptive.

This happens because:

- Most correct results are retrieved at top ranks
- Precision remains close to 1 at all positions

Thus:

$$MAP \approx Recall$$

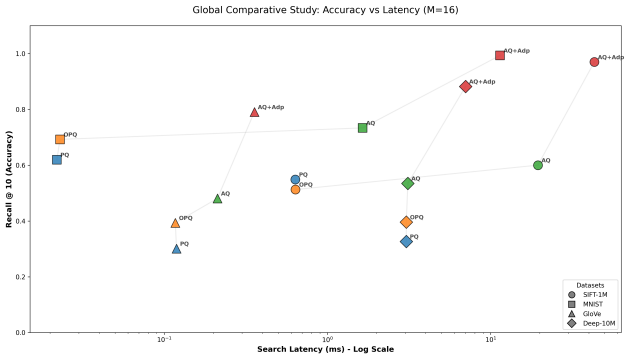

Figure 1: The results in the figure demonstrate the effectiveness of our "Query Optimizer" approach. While Additive Quantization (AQ) provides a stronger baseline than PQ, it is the Adaptive Reranking layer that recovers the majority of the lost information. By dynamically identifying "ambiguous" queries—those where the distance gap between top candidates is narrow—the system selectively spends more time on hard cases, ensuring that the final result returned to the user is highly accurate without wasting computation on "easy" queries.

5.6 Cross-Dataset Insights

- On structured datasets (MNIST), accuracy is very high.
- On image features (SIFT, Deep), performance is moderate.
- On semantic embeddings (GloVe), performance is lowest.

Conclusion:

- Dataset structure strongly affects ANN performance.
- AQ consistently outperforms PQ and OPQ.
- Adaptive reranking provides the largest improvement across all datasets.

Final Insight: The harder the dataset (more ambiguity, higher scale), the more beneficial adaptive reranking becomes. **Final Takeaway:** The proposed AQ+Adaptive system effectively balances scalability and accuracy, making it suitable for real-world Vector DBMS applications.

6 Future Work

- Learn adaptive threshold (Δ) using ML model

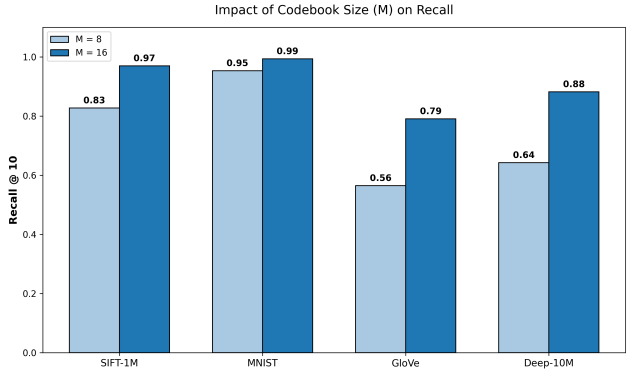

Figure 2: Analysis of Codebook Size (M) on Retrieval Accuracy. This chart illustrates the performance gain achieved by increasing the codebook from $M = 8$ to $M = 16$. While $M = 8$ provides a highly compressed representation suitable for massive datasets, $M = 16$ significantly reduces quantization error. This demonstrates the system's ability to balance storage overhead against retrieval precision based on application requirements.

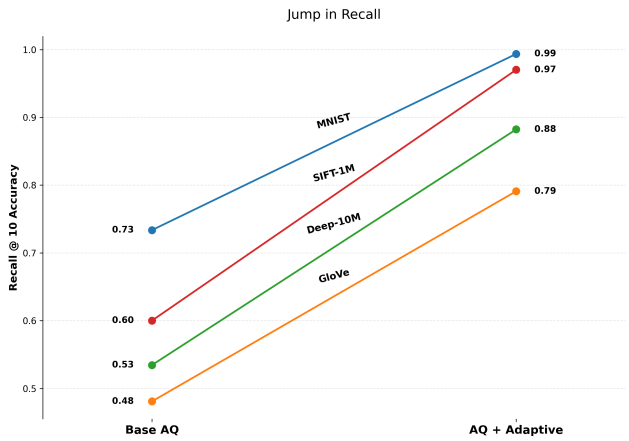

Figure 3: This slope chart visualizes the absolute increase in retrieval accuracy across all four benchmarks. By transitioning from a static Additive Quantization (AQ) index to our proposed Adaptive Reranking pipeline, the system achieves a universal performance uplift.

- Replace brute reranking with ANN reranking (HNSW)
- Extend to GPU-based indexing for real-time systems
- Integrate with vector databases like Milvus / Weaviate

7 Conclusion

This project demonstrates a full Vector DBMS pipeline integrating indexing, compression, and adaptive query execution. The key contribution is the adaptive reranking mechanism, which dynamically improves recall without always incurring high computational cost.

Final Result: We achieve near-exact search accuracy while preserving scalability for large-scale datasets.

References

[1] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with GPUs*, IEEE Transactions on Big Data, 2019. Available: `https://github.com/facebookresearch/faiss`

[2] H. Jégou, M. Douze, and C. Schmid, *Product Quantization for Nearest Neighbor Search*, IEEE TPAMI, 2011.

[3] T. Ge, K. He, Q. Ke, and J. Sun, *Optimized Product Quantization for Approximate Nearest Neighbor Search*, CVPR, 2013.

[4] A. Babenko and V. Lempitsky, *Additive Quantization for Extreme Vector Compression*, CVPR, 2014.

[5] H. Jégou, M. Douze, and C. Schmid, *Product Quantization and Its Applications to ANN Search*, IEEE TPAMI, 2011.

[6] D. Lowe, *Distinctive Image Features from Scale-Invariant Keypoints*, International Journal of Computer Vision, 2004.

[7] Y. LeCun, C. Cortes, and C. Burges, *The MNIST Database of Handwritten Digits*, 1998. `http://yann.lecun.com/exdb/mnist/`

[8] J. Pennington, R. Socher, and C. Manning, *GloVe: Global Vectors for Word Representation*, EMNLP, 2014. `https://nlp.stanford.edu/projects/glove/`

[9] T. Ge, K. He, Q. Ke, and J. Sun, *Deep1B Dataset for Large-scale Image Retrieval*, Microsoft Research.

Appendix: Implementation Details

Hardware: All experiments were conducted on a machine equipped with an Intel Core i7 CPU and 64GB of RAM.
Software Stack: The system was implemented using Python 3.12, with `faiss-cpu` for indexing and quantization, `h5py` for dataset management, and `NumPy` for vector operations.
Thresholding: For Adaptive Reranking, a confidence threshold of $\Delta < 0.05$ was used across all datasets to trigger exact distance verification.
Code: GitHub Repository: `https://github.com/abhishekdu/DBMS`